

Minor Project – Finite Element Method



Course Coordinator: Dr. Sandeep Singh

Course Code: ME 436/736

Name: Abhishek Yadav

Roll no.: 2502103031

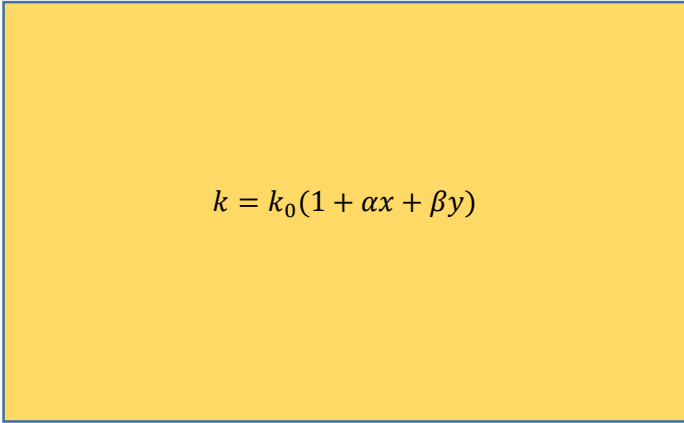
Department: Mechanical Engineering

Discipline: Thermal Energy Systems

Problem Statement:

Finite element model of 2D Steady State Heat Conduction in a rectangular plate with spatially varying thermal conductivity and mixed boundary conditions.

Domain:

$$T_{\infty} = 300K, h = \frac{25W}{m^2 K}$$

$$T = 400K$$
$$k = k_0(1 + \alpha x + \beta y)$$
$$T_{\infty} = 300K$$
$$h = 25 \left(\frac{W}{m^2 K} \right)$$
$$\frac{\partial T}{\partial \eta} = 0$$

where $k_0 = 200 \left(\frac{W}{m^2 K} \right)$, $\alpha = 0.1$ and $\beta = 0.05$ (Aluminium like material).

Governing Differential Equation:

$$\frac{\partial}{\partial x} \left(k(x, y) \left(\frac{\partial T}{\partial x} \right) \right) + \frac{\partial}{\partial y} \left(k(x, y) \left(\frac{\partial T}{\partial y} \right) \right) = 0$$

Developing Weak Form:

$$\iint \delta T \left(\frac{\partial}{\partial x} \left(k \left(\frac{\partial T}{\partial x} \right) \right) + \frac{\partial}{\partial y} \left(k \left(\frac{\partial T}{\partial y} \right) \right) \right) dx dy = 0$$

Taking the first term (A1) from above equation and using integration by parts, we get

$$\begin{aligned}
A1 &= \iint \delta T \left(\frac{\partial}{\partial x} k \left(\frac{\partial T}{\partial x} \right) \right) dx dy = \int_{y_1}^{y_2} \delta T \left(k \left(\frac{\partial T}{\partial x} \right)_{x_1}^{x_2} dy \right) - \int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{d(\delta T)}{dx} k \left(\frac{\partial T}{\partial x} \right) dx dy \\
&= \int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{\partial}{\partial x} \left(\delta T k \left(\frac{\partial T}{\partial x} \right) \right) dx dy - \int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{d(\delta T)}{dx} k \left(\frac{\partial T}{\partial x} \right) dx dy
\end{aligned}$$

Similarly second term (A2) can be written as,

$$A2 = \int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{\partial}{\partial y} \left(\delta T k \left(\frac{\partial T}{\partial y} \right) \right) dx dy - \int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{d(\delta T)}{dy} k \left(\frac{\partial T}{\partial y} \right) dx dy$$

Now combining A1 and A2 terms,

$$\begin{aligned}
&\left(\int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{\partial}{\partial x} \left(\delta T k \left(\frac{\partial T}{\partial x} \right) \right) + \frac{\partial}{\partial y} \left(\delta T k \left(\frac{\partial T}{\partial y} \right) \right) dx dy \right) \\
&\quad - \left(\int_{y_1}^{y_2} \int_{x_1}^{x_2} \left(\frac{d(\delta T)}{dx} k \left(\frac{\partial T}{\partial x} \right) + \frac{d(\delta T)}{dy} k \left(\frac{\partial T}{\partial y} \right) \right) dx dy \right) = 0
\end{aligned}$$

Using green's theorem,

$$\int_{y_1}^{y_2} \int_{x_1}^{x_2} \frac{\partial}{\partial x} \left(\delta T k \left(\frac{\partial T}{\partial x} \right) \right) + \frac{\partial}{\partial y} \left(\delta T k \left(\frac{\partial T}{\partial y} \right) \right) dx dy = \oint \delta T \left(k \left(\frac{\partial T}{\partial x} \right) \eta_x + k \left(\frac{\partial T}{\partial y} \right) \eta_y \right) dS$$

Therefore, the weak form equation can be written as,

$$\int [\nabla(\delta T)]^T (k \nabla T) \mathbf{dx dy} = \oint \delta T (k \nabla T) \boldsymbol{\eta} dS$$

LHS represents stiffness matrix due to heat conduction through area and RHS represents heat flux through boundary edges. It's also called load vector.

To make above weak form more general, we need to include the convection boundary condition.

$$k \nabla T \boldsymbol{\eta} = -h(T - T_{\infty})$$

Substituting this into the boundary integral of the above weak form(RHS), we get,

$$\oint \delta T (k \nabla T) \boldsymbol{\eta} dS = \oint -\delta T (h(T - T_{\infty})) dS$$

$$= - \oint \delta T h T dS + \oint \delta T h T_{\infty} dS$$

Therefore the final weak form is,

$$\int [\nabla(\delta T)]^T (k \nabla T) \, dx \, dy + \oint \delta T h T dS = \oint \delta T h T_{\infty} dS$$

Elemental Equation:

We will be using isoparametric element. Therefore, field variable can be interpolated as,

$$T = N_1 T_1 + N_2 T_2 + N_3 T_3 = [N][d_e]$$

and geometry can also be interpolated as,

$$x = N_1 x_1 + N_2 x_2 + N_3 x_3 = [N][x]$$

$$y = N_1 y_1 + N_2 y_2 + N_3 y_3 = [N][y]$$

Gradient of the T can be found as,

$$\nabla T = \begin{bmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial x} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_2}{\partial y} & \frac{\partial N_3}{\partial y} \end{bmatrix} [d_e] = [B][d_e]$$

From weak form equation,

$$\nabla(\delta T) = [B]\delta[d_e]$$

Therefore, the LHS of weak form equation becomes,

$$\int \delta[d_e]^T [B]^T k [B] [d_e] \, dx \, dy = \delta[d_e] \left(\int [B]^T k [B] \, dx \, dy \right) [d_e]$$

Where,

$$[K_e] = \int [B]^T k [B] \, dx \, dy = [B]^T [B] \int k(x, y) \, dx \, dy$$

[B] matrix is constant since triangular element is used. K_e is not constant but depends on the surface integral of $k(x, y)$.

Thermal conductivity for an triangular element can be given as,

$$k(x_e, y_e) = k_0(1 + \alpha x + \beta y)$$

where $x = [N][x]$ and $y = [N][y]$

Also RHS becomes,

$$\oint \delta T (k \nabla T) \cdot \eta dS = [d_e]^T \oint [N]^T q_\eta dS$$

where $[q_e] = \oint [N]^T q_\eta dS$ is the load vector for the element.

Therefore the Elemental equation becomes,

$$[K_e][d_e] = [q_e]$$

Interpolation function in x-y coordinate system:

$$N_1 = \frac{\alpha_1 + \beta_1 x + \gamma_1 y}{2A}$$

$$N_2 = \frac{\alpha_2 + \beta_2 x + \gamma_2 y}{2A}$$

$$N_3 = \frac{\alpha_3 + \beta_3 x + \gamma_3 y}{2A}$$

where A= area of triangular element

$$\alpha_1 = x_2 y_3 - x_3 y_2$$

$$\alpha_2 = -x_1 y_3 + x_3 y_1$$

$$\alpha_3 = x_1 y_2 - x_2 y_1$$

$$\beta_1 = y_2 - y_3$$

$$\beta_2 = y_3 - y_1$$

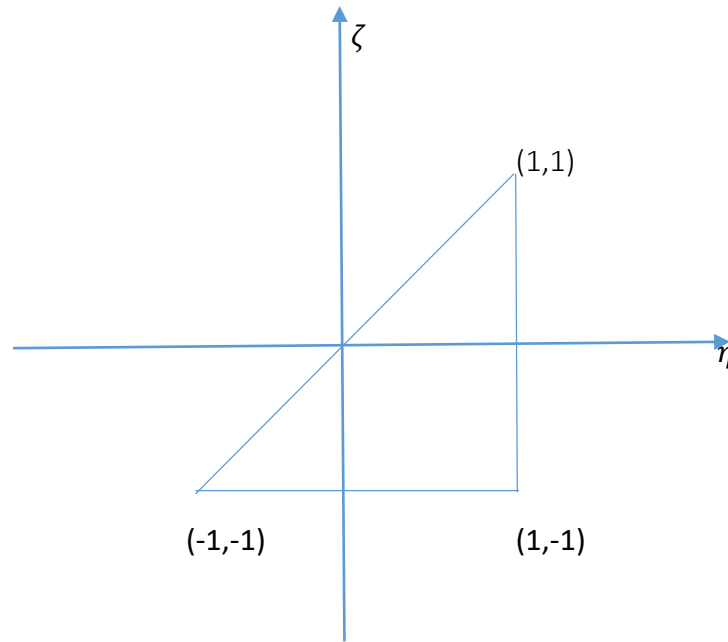
$$\beta_3 = y_1 - y_2$$

$$\gamma_1 = x_3 - x_2$$

$$\gamma_2 = x_1 - x_3$$

$$\gamma_3 = x_2 - x_1$$

Transforming the element to Natural Coordinate system:



Interpolation function in natural coordinate system can be found by assuming

$$x = a_0 + a_1\zeta + a_2\eta$$

$$y = b_0 + b_1\zeta + b_2\eta$$

Now substituting x and y values and corresponding ζ and η values we get interpolation function as,

$$H_1 = \frac{1 - \zeta}{2}$$

$$H_2 = \frac{\zeta - \eta}{2}$$

$$H_3 = \frac{1 + \eta}{2}$$

Therefore,

$$x = H_1x_1 + H_2x_2 + H_3x_3$$

$$y = H_1y_1 + H_2y_2 + H_3y_3$$

and Jacobian matrix can be written as,

$$J = 0.5 \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}$$

Therefore the elemental stiffness matrix can be found as,

$$K_e = k_0[B]^T \left(\int (1 + \alpha[H][x] + \beta[H][y]) |J| d\zeta d\eta \right) [B]$$

Where $-1 \leq \zeta \leq 1$ and $-1 \leq \eta \leq 1$

The order of polynomial inside the surface integral will be 1, so only 1 gauss point is required which is going to be $(\frac{1}{3}, -\frac{1}{3})$ and weight= 2.

For every element, K_e will change depending upon the coordinates of the vertices. The integral

$$\int_{-1}^1 \int_{-1}^1 [H] |J| d\zeta d\eta = \left[\frac{2}{3} \frac{2}{3} \frac{2}{3} \right] |J|$$

Applying convection boundary condition:

Note that convection boundary condition is applied to only the top and right boundary edges, as convection stiffness becomes zero for the all other edges. Insulated boundary condition does not contribute to the stiffness and load vector.

The boundary edges can be treated as linear element, the shape functions will be

$$N_1 = \frac{1 - \zeta}{2}$$

$$N_2 = \frac{1 + \zeta}{2}$$

Using the final weak form equation,

$$\int [\nabla(\delta T)]^T (k \nabla T) d\mathbf{x} d\mathbf{y} + \oint \delta T h T dS = \oint \delta T h T_\infty dS$$

2nd term on the LHS acts as convection stiffness. This can be converted into matrix form as

$$K_h = \oint h [N]^T [N] dS$$

After converting into natural coordinate system, we get

$$K_h = \oint h \begin{bmatrix} N_1^2 & N_1 N_2 \\ N_1 N_2 & N_2^2 \end{bmatrix} \frac{L_e}{2} d\zeta$$

After integration we get,

$$K_h = \frac{h L_e}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

This the convection stiffness matrix for the top and right boundary edge.

The RHS of the above weak form can be written as,

$$F_h = \oint h [N]^T T_\infty dS$$

```
K = 231x231
```

200.7500	-100.4167	0	0	0	0	0	0 ...
-100.4167	402.5833	-100.9167	0	0	0	0	0
0	-100.9167	404.5833	-101.4167	0	0	0	0
0	0	-101.4167	406.5833	-101.9167	0	0	0
0	0	0	-101.9167	408.5833	-102.4167	0	0
0	0	0	0	-102.4167	410.5833	-102.9167	0
0	0	0	0	0	-102.9167	412.5833	-103.4167
0	0	0	0	0	0	-103.4167	414.5833
0	0	0	0	0	0	0	-103.9167
0	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.

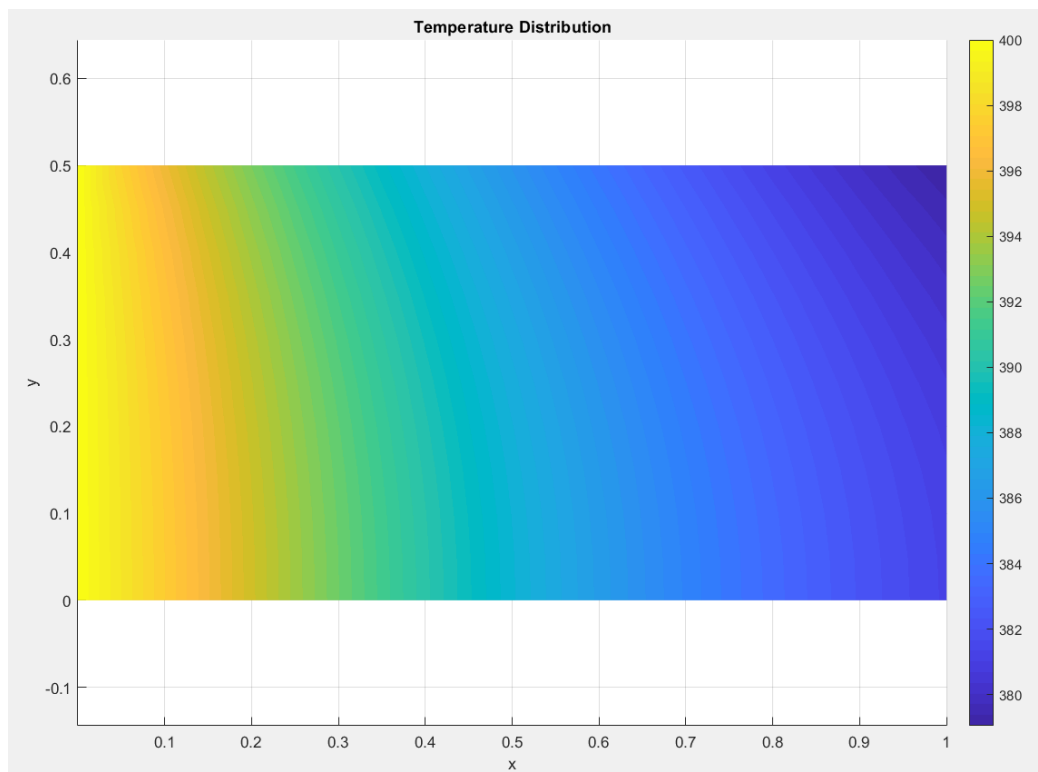
And global load vector obtained will be,

$$F = 231 \times 10^4 \times \begin{pmatrix} 0.0400 \\ 4.0167 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Temperature vector obtained will be,

```
T = 231x1
    400.0000
    398.6871
    397.3892
    396.1142
    394.8693
    393.6602
    392.4918
    391.3677
    390.2905
    389.2621
    .
    .
```

Temperature distribution:



Total heat transfer from the top + right boundary edge is obtained as: 3180.5 W

Maximum Temperature: 400K

Minimum Temperature: 379

MATLAB code:

```
clc;
clear;
close all;

% 1. Parameters

L = 1.0; % Plate length (m)
H = 0.5; % Plate height (m)

nx = 20; % Number of divisions in x
ny = 10; % Number of divisions in y

k0 = 200; % Base thermal conductivity
alpha = 0.1; % Conductivity variation in x
beta = 0.05; % Conductivity variation in y

h = 25; % Convection coefficient
T_inf = 300; % Ambient temperature
T_left = 400; % Left edge temperature

tol = 1e-12;

% 2. Generating Nodes

dx = L/nx;
dy = H/ny;

nodes = [];

for j = 0:ny

    for i = 0:nx
```

```

        x = i*dx;
        y = j*dy;

        nodes = [nodes; x y];

    end

end

nodes
num_nodes = size(nodes,1)

% 3. Generating triangular elements

elements = [];

for j = 1:ny

    for i = 1:nx

        n1 = (j-1)*(nx+1) + i;
        n2 = n1 + 1;

        n4 = j*(nx+1) + i;
        n3 = n4 + 1;

        % Triangle 1
        elements = [elements; n1 n2 n3];

        % Triangle 2
        elements = [elements; n1 n3 n4];

    end

end

elements
num_elements = size(elements,1)

```

```

% 4. Plotting triangular mesh

figure

triplot(elements, nodes(:,1), nodes(:,2), 'b')

axis equal

xlabel('x')
ylabel('y')

title('Triangular Mesh')

% 5. Initialize the global matrices

K = zeros(num_nodes,num_nodes);

F = zeros(num_nodes,1);

% 6. Element loops

for e = 1:num_elements

    element_nodes = elements(e,:); % element nodes

    % Coordinates of nodes
    x1 = nodes(element_nodes(1),1);
    y1 = nodes(element_nodes(1),2);

    x2 = nodes(element_nodes(2),1);
    y2 = nodes(element_nodes(2),2);

    x3 = nodes(element_nodes(3),1);

```

```

y3 = nodes(element_nodes(3),2);

% Jacobian Matrix

dx_dxi = (-0.5*x1) + (0.5*x2);
dy_dxi = (-0.5*y1) + (0.5*y2);

dx_deta = (-0.5*x2) + (0.5*x3);
dy_deta = (-0.5*y2) + (0.5*y3);

J = [dx_dxi  dy_dxi;
     dx_deta  dy_deta];

detJ = det(J);

invJ = inv(J);

% Shape function derivatives wrt xi and eta

dN_dxi = [-0.5; 0.5; 0.0];

dN_deta = [0.0; -0.5; 0.5];

% Derivatives wrt x and y [B]

gradN = invJ * [dN_dxi'; dN_deta'];

dN_dx = gradN(1,:);
dN_dy = gradN(2,:);

% B matrix

B = [dN_dx; dN_dy];

% Gauss Point

xi = 1/3;

```

```

eta = -1/3;

weight = 2;

% Shape functions at Gauss point

N1 = 0.5*(1 - xi);

N2 = 0.5*(xi - eta);

N3 = 0.5*(1 + eta);

N = [N1 N2 N3];

% Coordinates at Gauss point

x_gp = N*[x1; x2; x3];

y_gp = N*[y1; y2; y3];

% Spatially variable thermal conductivity

k = k0*(1 + alpha*x_gp + beta*y_gp);

% Element stiffness matrix

Ke = (B'*k*B)*detJ*weight;

% Assembly

for i = 1:3

    for j = 1:3

        K(element_nodes(i), element_nodes(j)) = K(element_nodes(i),
element_nodes(j)) + Ke(i,j);

```

```

        end

    end

end

% 7. Apply convection boundary condition

for e = 1:num_elements

    element_nodes = elements(e,:);

    edge_list = [1 2; 2 3; 3 1];

    for edge = 1:3

        local_nodes = edge_list(edge,:);

        n1 = element_nodes(local_nodes(1));
        n2 = element_nodes(local_nodes(2));

        p1 = nodes(n1,:);
        p2 = nodes(n2,:);

        % TOP EDGE

        if abs(p1(2)-H)<tol && abs(p2(2)-H)<tol

            Le = norm(p2-p1);

            Kh = h*Le/6 * [2 1; 1 2];

            Fh = h*T_inf*Le/2 * [1; 1];

            edge_nodes = [n1 n2];

```

```

        K(edge_nodes,edge_nodes) = K(edge_nodes,edge_nodes) + Kh;

        F(edge_nodes) = F(edge_nodes) + Fh;

    end

    % RIGHT EDGE

    if abs(p1(1)-L)<tol && abs(p2(1)-L)<tol

        Le = norm(p2-p1);

        Kh = h*Le/6 * [2 1; 1 2];

        Fh = h*T_inf*Le/2 * [1; 1];

        edge_nodes = [n1 n2];

        K(edge_nodes,edge_nodes) = K(edge_nodes,edge_nodes) + Kh;

        F(edge_nodes) = F(edge_nodes) + Fh;

    end

end

end

K
F

% 8. Apply dirichlet boundary condition

left_nodes = [];

for i = 1:num_nodes

```

```

    if abs(nodes(i,1)) < tol

        left_nodes = [left_nodes; i];

    end

end

for i = 1:length(left_nodes)

    node = left_nodes(i);

    F = F - K(:,node)*T_left;

    K(node,:) = 0;
    K(:,node) = 0;

    K(node,node) = 1;

    F(node) = T_left;

end
F

% 9. Solving system of equations

T = K\F;
T

% 10. Temperature contour plot

figure

trisurf(elements, nodes(:,1), nodes(:,2), T, 'EdgeColor','none')

view(2)

axis equal

```

```

xlabel('x')
ylabel('y')

title('Temperature Distribution')

colorbar

shading interp

% 11. Post Processing and results

fprintf('Maximum Temperature = %.2f K\n',max(T));

fprintf('Minimum Temperature = %.2f K\n',min(T));

% 12. Boundary Heat Flux Calculation

Q_total = 0;

fprintf('\nBoundary Heat Flux Results:\n');

for e = 1:num_elements

    element_nodes = elements(e,:);

    edge_list = [1 2;
                 2 3;
                 3 1];

    for edge = 1:3

        local_nodes = edge_list(edge,:);

        n1 = element_nodes(local_nodes(1));
        n2 = element_nodes(local_nodes(2));

```

```

p1 = nodes(n1,:);
p2 = nodes(n2,:);

% Top edge convection

if abs(p1(2)-H)<tol && abs(p2(2)-H)<tol

    Le = norm(p2-p1); % edge length

    % 1-point Gauss quadrature
    xi = 0;
    weight = 2;

    % Shape functions at Gauss point
    N1 = 0.5*(1-xi);
    N2 = 0.5*(1+xi);

    Nedge = [N1 N2];

    Te = [T(n1); T(n2)]; % Edge nodal temperatures

    T_gp = Nedge*Te; % Temperature at Gauss point

    q = h*(T_gp - T_inf); % Boundary heat flux

    J_edge = Le/2; % Jacobian for line element

    Qe = q*J_edge*weight; % Heat transfer through edge

    Q_total = Q_total + Qe; % Total heat transfer

    fprintf('Top Edge (%d-%d): q = %.4f W/m^2\n', n1,n2,q);

end

% Right convection edge

if abs(p1(1)-L)<tol && abs(p2(1)-L)<tol

```

```

    Le = norm(p2-p1);

    xi = 0;
    weight = 2;

    N1 = 0.5*(1-xi);
    N2 = 0.5*(1+xi);

    Nedge = [N1 N2];

    Te = [T(n1); T(n2)];

    T_gp = Nedge*Te;

    q = h*(T_gp - T_inf);

    J_edge = Le/2;

    Qe = q*J_edge*weight;

    Q_total = Q_total + Qe;

    fprintf('Right Edge (%d-%d): q = %.4f W/m^2\n', n1,n2,q);

end

end

end

fprintf('\nTotal Convective Heat Transfer = %.4f W\n',Q_total);

```